Experiment 2: Interface LEDs, Push Buttons, Buzzers, Temperature and Humidity Sensors Using ESP32 (Wokwi Simulation)

Aim

To interface an **LED**, **Push Button**, **Buzzer**, and **DHT22 Temperature & Humidity Sensor** with the ESP32 using the **Wokwi Simulator** and display the temperature and humidity values on the Serial Monitor.

Objectives

- Interface digital input and output devices with ESP32.
- Control an LED using a push button.
- Generate sound using a buzzer.
- Read temperature and humidity using a DHT22 sensor.
- Display sensor readings on the Serial Monitor.

Apparatus Required

S.No	Components	Quantity
1	ESP32 Dev Module	1
2	LED	1
3	220 Ω Resistor	1
4	Push Button	1
5	Active Buzzer	1
6	DHT22 Temperature & Humidity Sensor	1
7	Breadboard	1
8	Jumper Wires	As required
9	Computer with Arduino IDE / Wokwi	1

Software Required

- Arduino IDE
- ESP32 Board Package
- Wokwi Simulator
- DHT Sensor Library

Theory

ESP32

ESP32 is a powerful 32-bit microcontroller with built-in Wi-Fi and Bluetooth. It supports digital and analog input/output for interfacing with sensors and actuators.

LED

A Light Emitting Diode (LED) is used as a visual indicator. It glows when the GPIO pin outputs HIGH.

Push Button

A push button is a digital input device. When pressed, it changes the logic level, allowing the ESP32 to detect user input.

Buzzer

A buzzer converts electrical signals into sound and is commonly used for alarms and notifications.

DHT22 Sensor

The DHT22 measures:

- Temperature (°C)
- Relative Humidity (%)

It communicates with the ESP32 using a single digital data pin.

Circuit Connections

LED

LED	ESP32
Anode (+)	GPIO 2 (through 220 Ω resistor)
Cathode (-)	GND

Push Button

Push Button	ESP32
One Terminal	GPIO 4
Other Terminal	GND

(Use the ESP32's internal pull-up resistor.)

Buzzer

Buzzer	ESP32
Positive	GPIO 15
Negative	GND

DHT22

DHT22	ESP32
VCC	3.3V
GND	GND
DATA	GPIO 18

Circuit Diagram (Text Representation)

```
                ESP32

GPIO2 -------- LED (+)
GND ---------- LED (-)

GPIO4 -------- Push Button
GND ---------- Push Button

GPIO15 ------- Buzzer (+)
GND ---------- Buzzer (-)

3.3V --------- DHT22 VCC
GPIO18 ------- DATA
GND ---------- GND
```

Algorithm

1. Start the program.
2. Initialize the Serial Monitor.
3. Configure LED and buzzer pins as OUTPUT.
4. Configure the push button pin as INPUT_PULLUP.
5. Initialize the DHT22 sensor.
6. Read the push button state.

7. If the button is pressed:
    - Turn ON the LED.
    - Turn ON the buzzer.
8. Otherwise:
    - Turn OFF the LED.
    - Turn OFF the buzzer.
9. Read the temperature and humidity from the DHT22 sensor.
10. Display the values on the Serial Monitor.
11. Repeat continuously.

Arduino Program

```
#include <DHT.h>

#define DHTPIN 18
#define DHTTYPE DHT22

#define LED_PIN 2
#define BUTTON_PIN 4
#define BUZZER_PIN 15

DHT dht(DHTPIN, DHTTYPE);

void setup() {

  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  pinMode(BUTTON_PIN, INPUT_PULLUP);

  dht.begin();
}

void loop() {

  int button = digitalRead(BUTTON_PIN);

  if(button == LOW)
  {
    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
  }
  else
  {
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
  }

  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  Serial.print("Temperature : ");
  Serial.print(temperature);
```

```
  Serial.println(" °C");

  Serial.print("Humidity : ");
  Serial.print(humidity);
  Serial.println(" %");

  Serial.println("------------------------");

  delay(2000);
}
```

Procedure (Using Wokwi Simulator)

Step 1: Open Wokwi

1. Open your web browser.
2. Go to **Wokwi Simulator**.
3. Click **New Project**.
4. Select **ESP32 Dev Module**.

Step 2: Add Components

Click the **+ (Add Part)** button and add:

- ESP32 Dev Module
- LED
- 220 Ω Resistor
- Push Button
- Active Buzzer
- DHT22 Sensor

Step 3: Connect the Components

Make the connections exactly as shown below.

Device	ESP32 Pin
LED	GPIO2
Push Button	GPIO4
Buzzer	GPIO15
DHT22 DATA	GPIO18
DHT22 VCC	3.3V
DHT22 GND	GND

Step 4: Copy the Program

1. Delete the default Arduino program.
2. Paste the program given above.
3. Save the project.

Step 5: Start the Simulation

1. Click ▶ **Start Simulation**.
2. The ESP32 program starts executing.
3. The Serial Monitor opens automatically.

Step 6: Test the Push Button

- Click the push button in Wokwi.

Observation:

- LED turns ON.
- Buzzer produces a sound.

Release the button.

Observation:

- LED turns OFF.
- Buzzer stops.

Step 7: Change Temperature

1. Click the DHT22 sensor.
2. Adjust the **Temperature** slider.

Example:

```
Temperature : 25°C
```

Increase it to **35°C**.

Serial Monitor:

```
Temperature : 35°C
```

Step 8: Change Humidity

Move the **Humidity** slider.

Example:

```
Humidity : 60%
```

Increase it to **80%**.

Serial Monitor:

```
Humidity : 80%
```

Step 9: Observe the Output

The Serial Monitor continuously displays:

- Temperature
- Humidity
- LED and buzzer operation based on the push button state

Sample Output

```
Temperature : 27.3 °C
Humidity : 58 %

-----------------------

Temperature : 30.1 °C
Humidity : 72 %

-----------------------

Temperature : 34.6 °C
Humidity : 81 %

-----------------------
```

Expected Output

- ✔ LED turns ON when the push button is pressed.
- ✔ Buzzer sounds when the push button is pressed.
- ✔ Temperature changes when the DHT22 temperature slider is adjusted.
- ✔ Humidity changes when the DHT22 humidity slider is adjusted.
- ✔ Sensor values are displayed correctly on the Serial Monitor.

Applications

- Smart Home Automation
- Temperature and Humidity Monitoring
- Security Alarm Systems
- Industrial Monitoring
- Weather Stations
- Smart Agriculture
- IoT Environmental Monitoring

Result

The LED, push button, buzzer, and DHT22 temperature and humidity sensor were successfully interfaced with the ESP32 using the Wokwi simulator. The LED and buzzer responded correctly to push button input, and the temperature and humidity readings were displayed successfully on the Serial Monitor.

Experiment 3: Interface Ultrasonic, PIR and Gas Sensors with ESP32 Using Wokwi Simulation

Aim

To interface the **HC-SR04 Ultrasonic Sensor**, **PIR Motion Sensor**, and **MQ-2 Gas Sensor** with the ESP32 using the **Wokwi Simulator** and display the sensor data on the Serial Monitor.

Procedure (Using Wokwi Simulator)

Step 1: Open Wokwi

1. Open your web browser.
2. Go to **Wokwi Simulator**.
3. Sign in (optional).

Step 2: Create a New Project

1. Click **New Project**.
2. Select **ESP32 Dev Module**.
3. A new ESP32 project opens with a sample program.

Step 3: Add Components

Click the **+ (Add Part)** button and add the following components:

- ESP32 Dev Module
- HC-SR04 Ultrasonic Sensor
- PIR Motion Sensor
- MQ-2 Gas Sensor (or use a potentiometer if MQ-2 is unavailable in Wokwi)
- Jumper Wires

Step 4: Make the Connections

HC-SR04

Sensor Pin	ESP32 Pin
VCC	5V
GND	GND
Trig	GPIO 5
Echo	GPIO 18

PIR Sensor

Sensor Pin	ESP32 Pin
VCC	5V
GND	GND
OUT	GPIO 19

MQ-2 Gas Sensor

Sensor Pin	ESP32 Pin
VCC	5V
GND	GND
AO	GPIO 34

Step 5: Enter the Program

1. Delete the default Arduino code.

2. Copy and paste the ESP32 program.
3. Save the project.

```
// Experiment No. 3
// Interfacing HC-SR04, PIR Sensor and MQ-2 Gas Sensor with ESP32

// Pin Definitions
#define TRIG_PIN 5
#define ECHO_PIN 18
#define PIR_PIN 19
#define GAS_PIN 34

long duration;
float distance;
int gasValue;
int pirState;

void setup() {

  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(PIR_PIN, INPUT);

  pinMode(GAS_PIN, INPUT);

  Serial.println("ESP32 Sensor Interfacing");
}

void loop() {

  // ---------- Ultrasonic Sensor ----------

  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);

  digitalWrite(TRIG_PIN, LOW);

  duration = pulseIn(ECHO_PIN, HIGH);

  distance = duration * 0.0343 / 2;

  // ---------- PIR Sensor ----------
```

```
  pirState = digitalRead(PIR_PIN);

  // ---------- MQ-2 Gas Sensor ----------

  gasValue = analogRead(GAS_PIN);

  // ---------- Display Output ----------

  Serial.println("------------------------------");

  Serial.print("Distance : ");
  Serial.print(distance);
  Serial.println(" cm");

  if (pirState == HIGH)
  {
   Serial.println("Motion Detected");
  }
  else
  {
   Serial.println("No Motion");
  }

  Serial.print("Gas Sensor Value : ");
  Serial.println(gasValue);

  delay(1000);
}
```

Step 6: Start the Simulation

1. Click the ▶ **Start Simulation** button.
2. The ESP32 starts executing the program.
3. The Serial Monitor opens automatically.

Step 7: Test the Ultrasonic Sensor

1. Click the HC-SR04 sensor.
2. Change the **Distance** value using the slider.
3. Observe that the displayed distance changes on the Serial Monitor.

Example:

```
Distance : 20 cm
```

Increase the distance to **100 cm**.

Serial Monitor:

```
Distance : 100 cm
```

Step 8: Test the PIR Sensor

1. Click the PIR sensor.
2. Toggle the **Motion** switch.

When Motion = ON

```
Motion Detected
```

When Motion = OFF

```
No Motion
```

Step 9: Test the MQ-2 Gas Sensor

1. Click the MQ-2 sensor (or adjust the potentiometer if used as a substitute).
2. Increase or decrease the analog output.

Example:

```
Gas Sensor Value : 180
```

Increase the gas level:

```
Gas Sensor Value : 920
```

Step 10: Observe the Output

The Serial Monitor continuously displays:

- Distance
- Motion Status
- Gas Sensor Reading

Sample Wokwi Output

```
--------------------------------
Distance : 32.5 cm
Motion Detected
Gas Sensor Value : 785

--------------------------------
Distance : 45.2 cm
No Motion
```

```
Gas Sensor Value : 250

--------------------------------
Distance : 18.7 cm
Motion Detected
Gas Sensor Value : 910
```

Expected Output

When the simulation runs successfully:

- ✔ Distance changes when the ultrasonic sensor slider is adjusted.
- ✔ Motion status changes when the PIR sensor is toggled.
- ✔ Gas sensor value changes when the MQ-2 output is varied.
- ✔ All readings are displayed on the Serial Monitor.

Result

The HC-SR04 Ultrasonic Sensor, PIR Motion Sensor, and MQ-2 Gas Sensor were successfully interfaced with the ESP32 using the Wokwi simulator. The sensor data was displayed correctly on the Serial Monitor, verifying the successful operation of the virtual embedded system.